

Francisco, France Gabriel R.

Code inside github

Use-Case: Map Navigation

A* uses:

- **$g(n)$** : the cost from the start node to the current node.
- **$h(n)$** : we'll use Manhattan distance
- **$f(n) = g(n) + h(n)$** : total estimated cost of the cheapest solution through node n.
-
- 0 = free space (walkable).
- 1 = wall/obstacle (blocked).
- This grid is the map where your pathfinding runs.

Example:

Start $(0, 0)$ is top-left, Goal $(4, 5)$ is bottom-right.

Heuristic Function (Manhattan Distance)

- This measures "straight-line distance" in a grid, only moving **up, down, left, right**.
- Guides A* to reach the goal faster.
- Without it, A* would act like Dijkstra's algorithm (slower).

. A* Algorithm

- Uses a **priority queue** (heapq) to always explore the **most promising path**.
- Tracks:
- $f = g + h \rightarrow$ estimated total cost
- g = steps taken so far
- h = heuristic estimate to the goal
- Expands neighbors until it finds the **shortest path**.
- Returns a **list of grid cells** from start $\rightarrow$ goal.

Visualization (Plotly)

- Draws the map:

- White squares = walkable cells
- Black squares = obstacles
- Draws the path (red line).
- Start = green marker, Goal = blue marker.

Saving Output

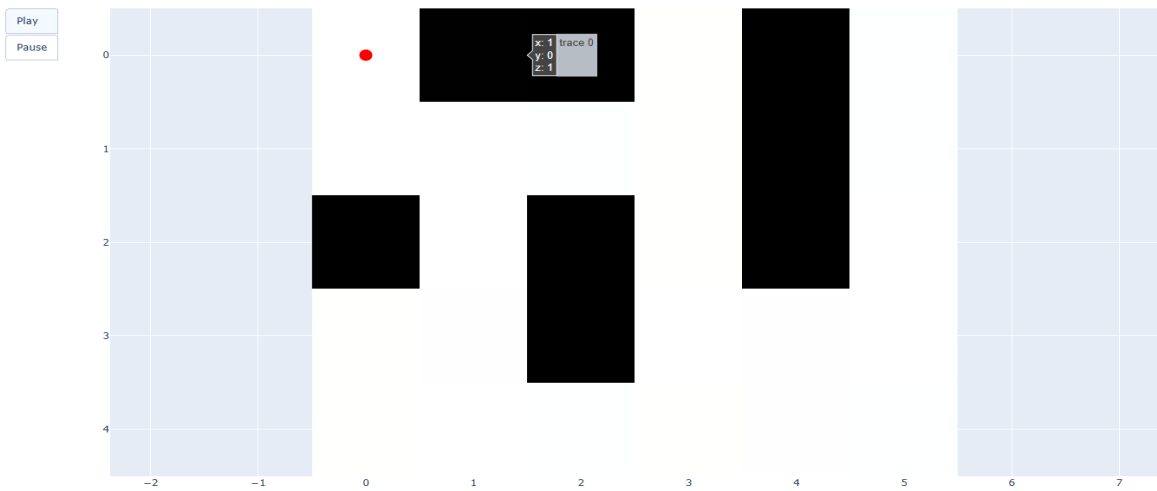
- .png → snapshot of the path
- .json → interactive Plotly data (you can reload/modify later)

Why is it useful?

1. **AI Pathfinding** → same algorithm used in games (NPC movement, robots).
2. **Visualization** → helps debug if the algorithm avoids obstacles correctly.
3. **Customizable** → you can change grid size, obstacles, or movement rules.

Visual representation screenshots:

A* Pathfinding with Moving Agent & Trail



A* Pathfinding with Moving Agent & Trail

